



BABU BANARASI DAS UNIVERSITY
SCHOOL OF COMPUTER APPLICATIONS

NO SQL PROJECT

Student Management System Using MongoDB

Submitted To	Submitted By
Mr. Harendra Singh Rajpoot	Diwakar Yadav
Department BCA (DS & AI)	Roll No. 34
Class BCA DS23	Academic Session 2026–2027
Date 25th September 2026	Project Student Management System Using MongoDB

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

1. CREATE DATABASE

The project starts by creating or switching to the Students database. This provides the database context in which the student collection is maintained.

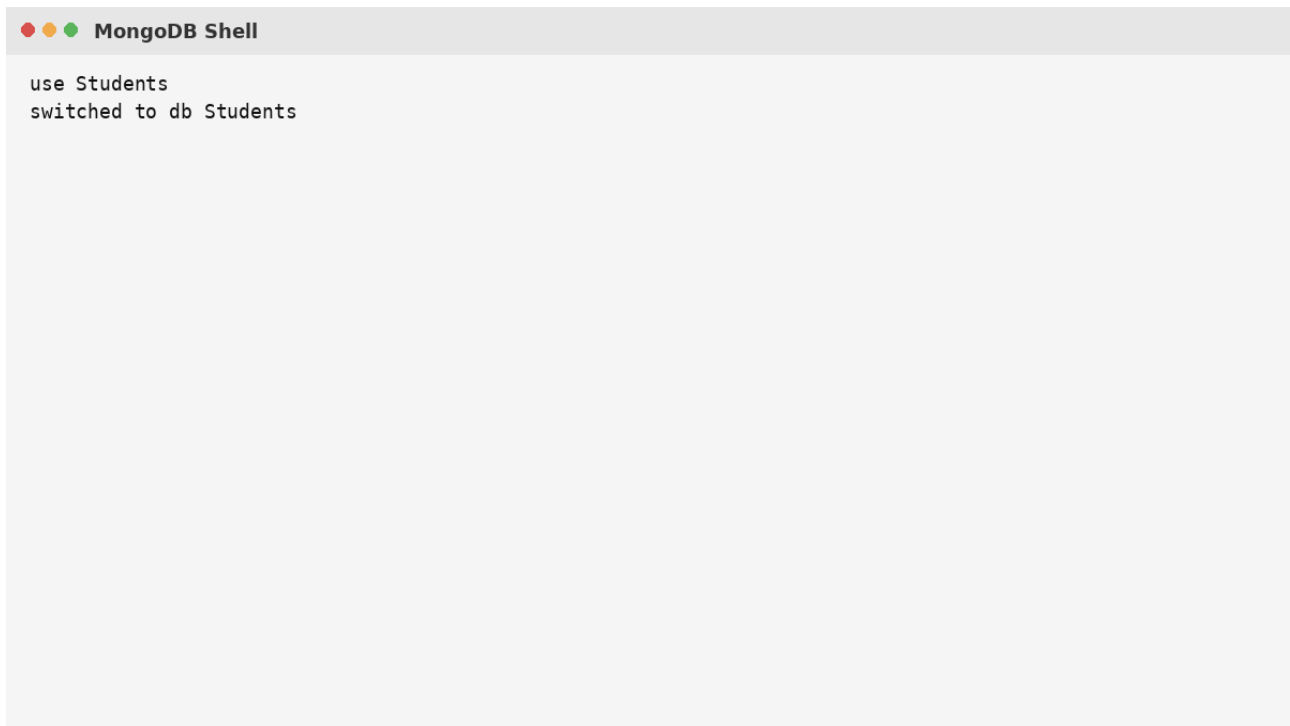
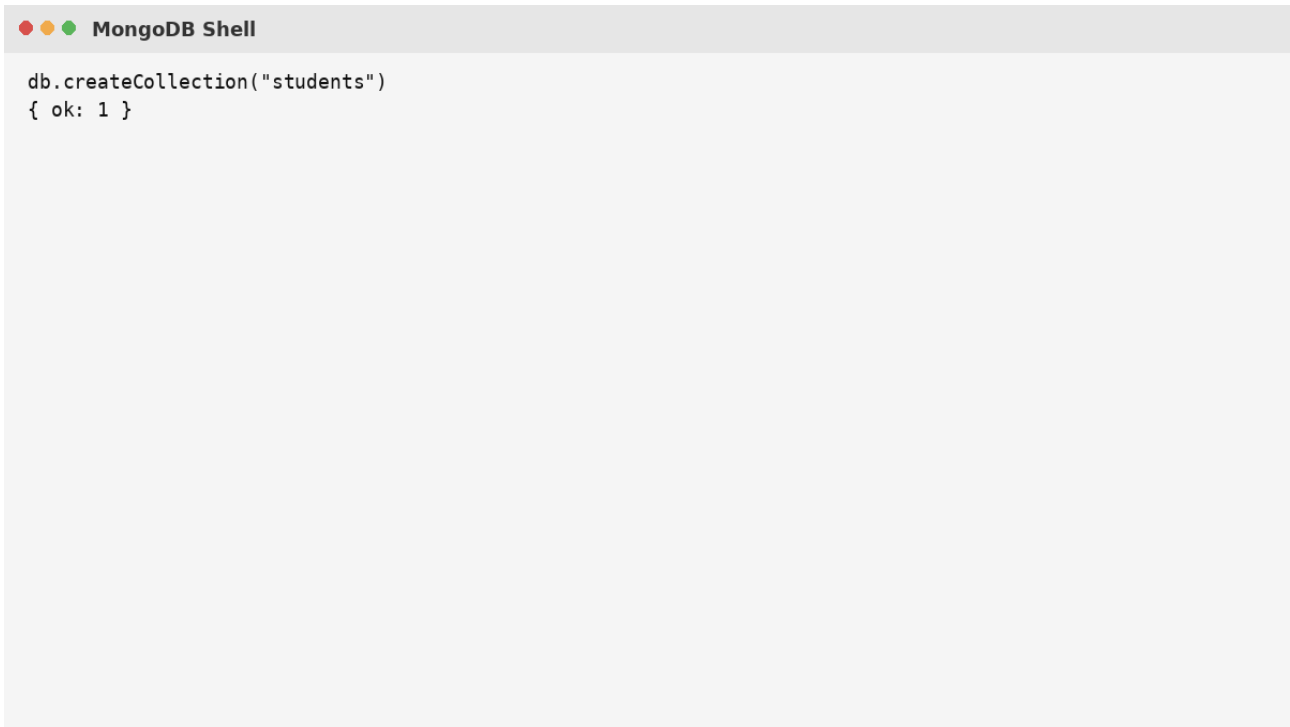


Figure: MongoDB execution screenshot — 1. CREATE DATABASE

2. CREATE COLLECTION

A collection named students is created to store the student documents.

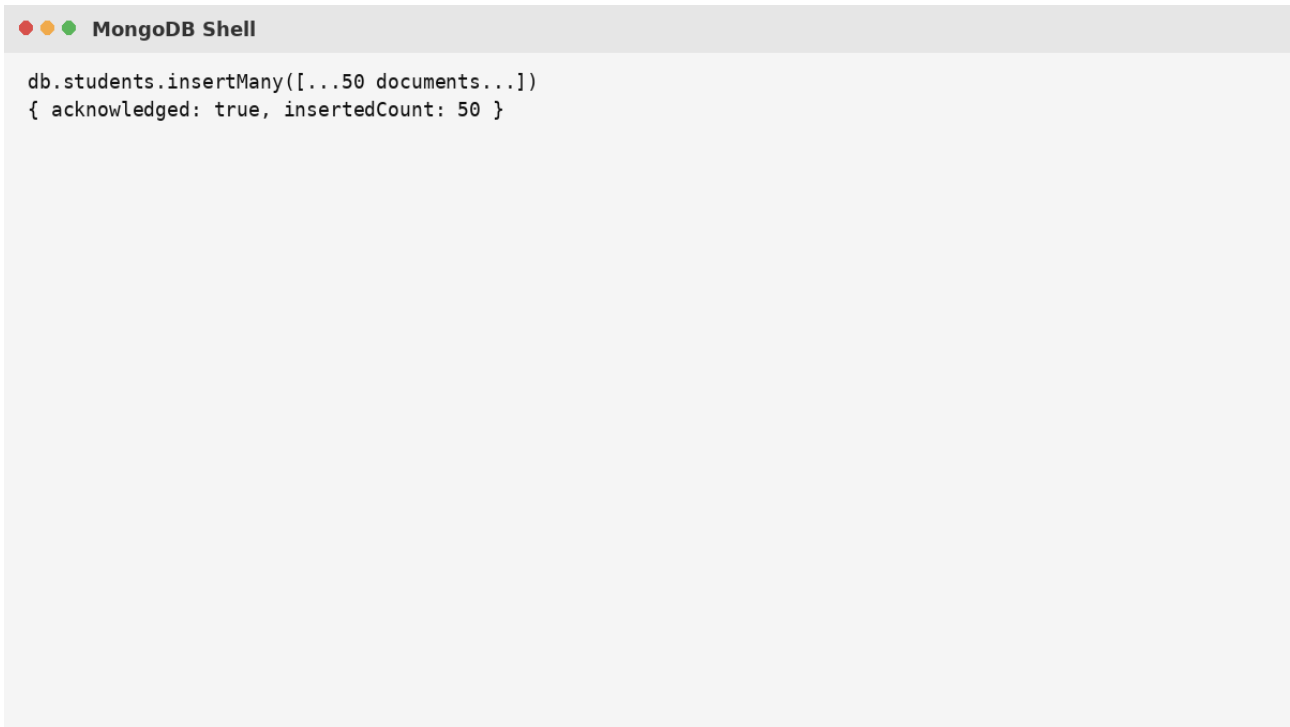
A screenshot of the MongoDB Shell interface. The title bar at the top shows three colored dots (red, yellow, green) followed by the text "MongoDB Shell". The main area of the window is light gray and contains two lines of text: the command `db.createCollection("students")` on the first line and the output `{ ok: 1 }` on the second line.

```
db.createCollection("students")
{ ok: 1 }
```

Figure: MongoDB execution screenshot — 2. CREATE COLLECTION

3. INSERT 50 STUDENTS

Multiple student documents are inserted into the students collection. The complete 50-record dataset is reproduced in the dataset section of this report.

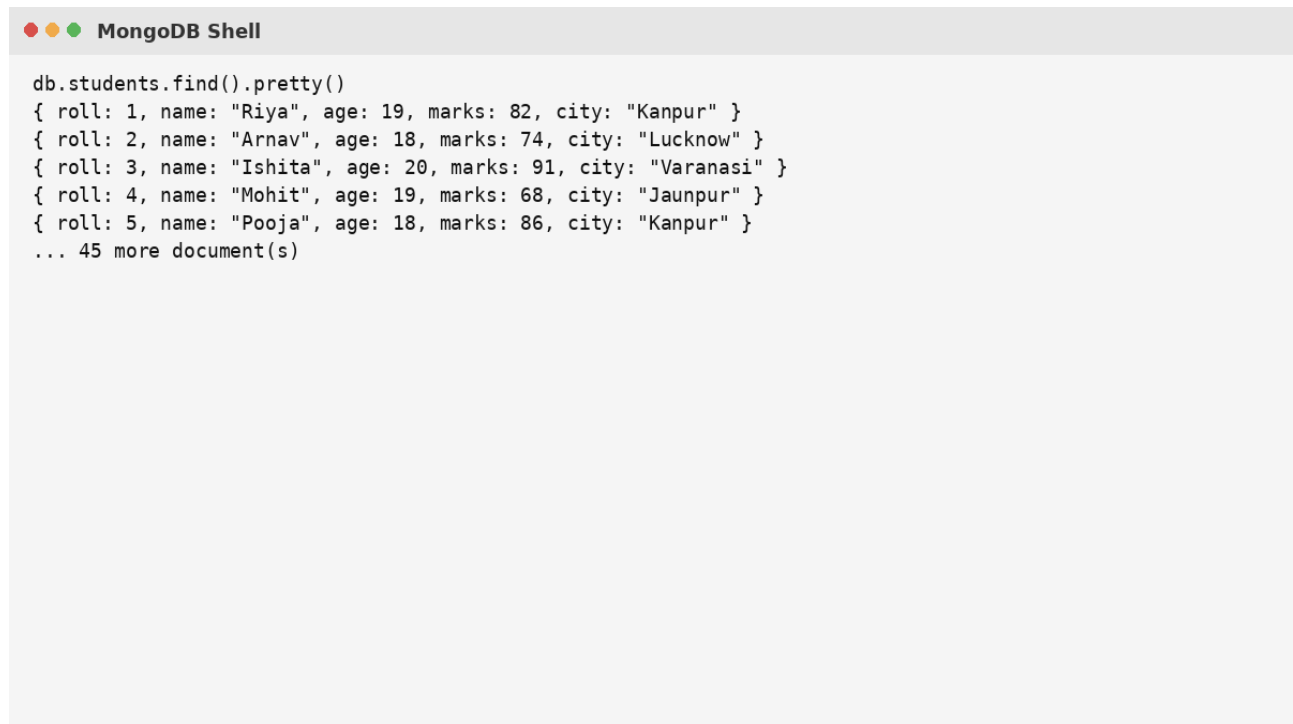
A screenshot of the MongoDB Shell interface. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "MongoDB Shell". The main area contains two lines of text: the command `db.students.insertMany([...50 documents...])` and the resulting JSON output `{ acknowledged: true, insertedCount: 50 }`.

```
db.students.insertMany([...50 documents...])
{ acknowledged: true, insertedCount: 50 }
```

Figure: MongoDB execution screenshot — 3. INSERT 50 STUDENTS

4. DISPLAY ALL STUDENTS

The display operation retrieves documents from the students collection and shows the stored student records.



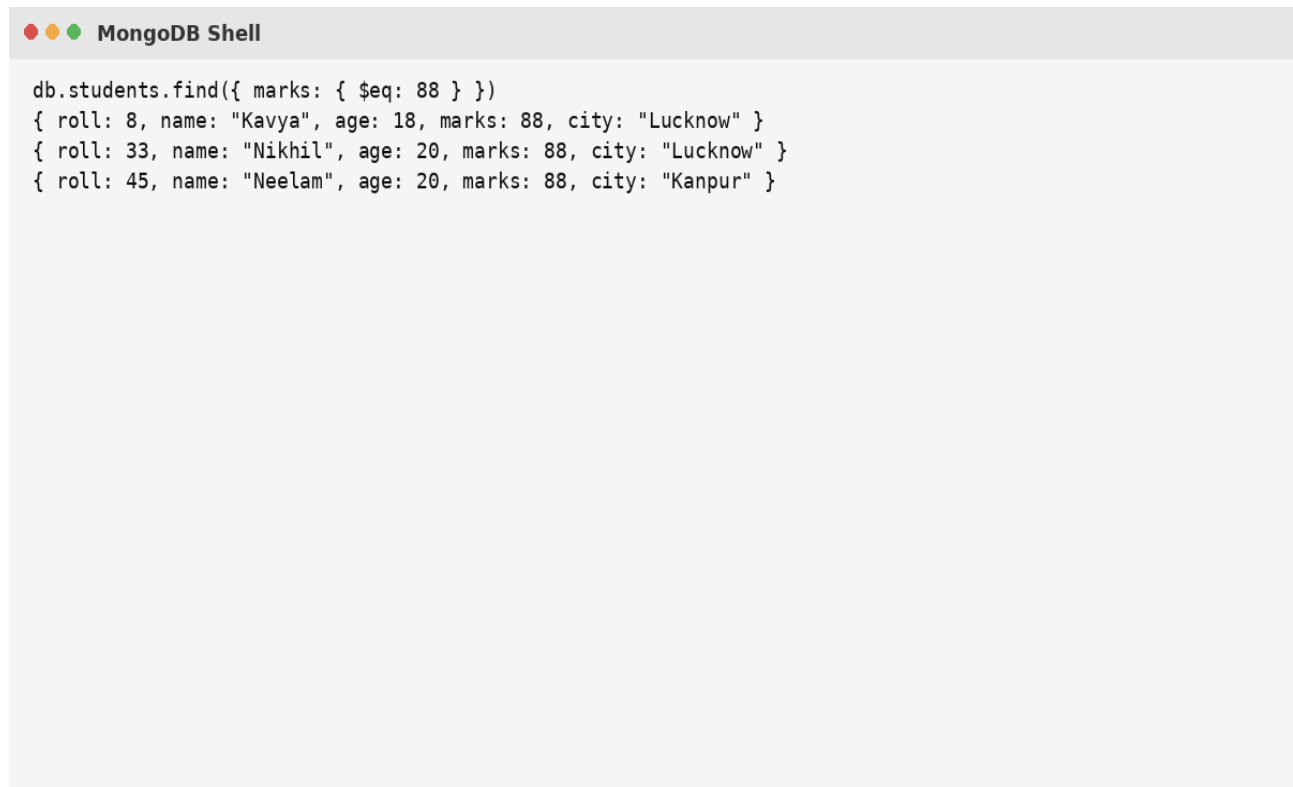
```
● ● ● MongoDB Shell

db.students.find().pretty()
{ roll: 1, name: "Riya", age: 19, marks: 82, city: "Kanpur" }
{ roll: 2, name: "Arnav", age: 18, marks: 74, city: "Lucknow" }
{ roll: 3, name: "Ishita", age: 20, marks: 91, city: "Varanasi" }
{ roll: 4, name: "Mohit", age: 19, marks: 68, city: "Jaunpur" }
{ roll: 5, name: "Pooja", age: 18, marks: 86, city: "Kanpur" }
... 45 more document(s)
```

Figure: MongoDB execution screenshot — 4. DISPLAY ALL STUDENTS

5. EQUAL TO — \$eq

The \$eq comparison operator finds students whose marks are exactly equal to 88.



A screenshot of the MongoDB Shell interface. The title bar shows three colored dots (red, yellow, green) followed by the text "MongoDB Shell". The main area displays a query and its results. The query is `db.students.find({ marks: { $eq: 88 } })`. The results are three JSON documents: `{ roll: 8, name: "Kavya", age: 18, marks: 88, city: "Lucknow" }`, `{ roll: 33, name: "Nikhil", age: 20, marks: 88, city: "Lucknow" }`, and `{ roll: 45, name: "Neelam", age: 20, marks: 88, city: "Kanpur" }`.

```
db.students.find({ marks: { $eq: 88 } })
{ roll: 8, name: "Kavya", age: 18, marks: 88, city: "Lucknow" }
{ roll: 33, name: "Nikhil", age: 20, marks: 88, city: "Lucknow" }
{ roll: 45, name: "Neelam", age: 20, marks: 88, city: "Kanpur" }
```

Figure: MongoDB execution screenshot — 5. EQUAL TO \$eq

6. GREATER THAN — \$gt

The \$gt comparison operator finds students whose marks are greater than 85.

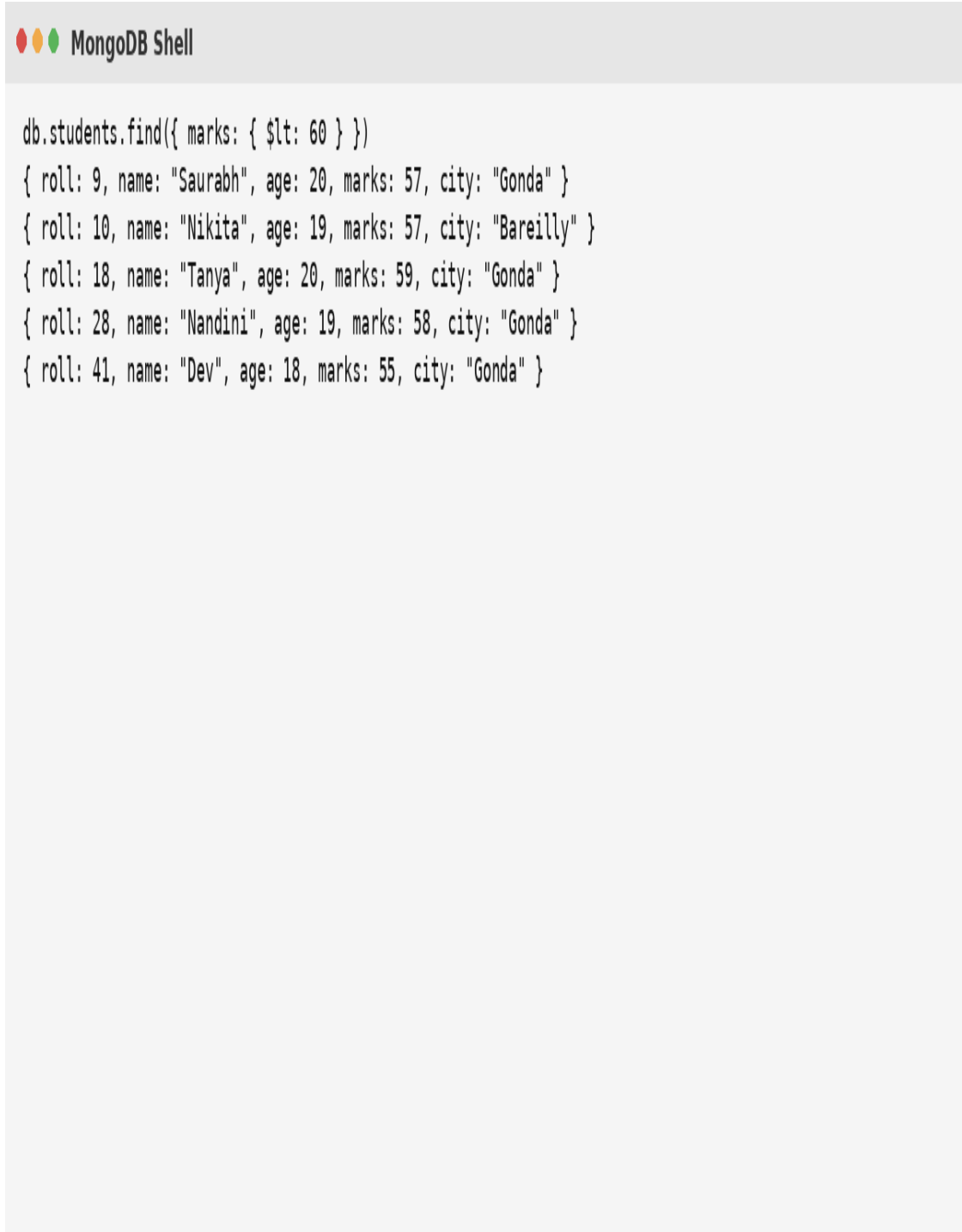
MongoDB Shell

```
db.students.find({ marks: { $gt: 85 } })
{ roll: 3, name: "Ishita", age: 20, marks: 91, city: "Varanasi" }
{ roll: 5, name: "Pooja", age: 18, marks: 86, city: "Kanpur" }
{ roll: 7, name: "Ritesh", age: 19, marks: 93, city: "Prayagraj" }
{ roll: 8, name: "Kavya", age: 18, marks: 88, city: "Lucknow" }
{ roll: 12, name: "Muskan", age: 20, marks: 95, city: "Kanpur" }
{ roll: 14, name: "Simran", age: 18, marks: 89, city: "Varanasi" }
{ roll: 16, name: "Aarohi", age: 19, marks: 92, city: "Prayagraj" }
... 12 more document(s)
```

Figure: MongoDB execution screenshot — 6. GREATER THAN \$gt

7. LESS THAN — \$lt

The \$lt comparison operator finds students whose marks are less than 60.

A screenshot of the MongoDB Shell interface. The title bar at the top says "MongoDB Shell" with three colored dots (red, orange, green) to its left. The main area contains a query and its results. The query is `db.students.find({ marks: { $lt: 60 } })`. Below it, five JSON documents are displayed, each representing a student with fields for roll, name, age, marks, and city. The marks for all five students are less than 60.

```
db.students.find({ marks: { $lt: 60 } })
{ roll: 9, name: "Saurabh", age: 20, marks: 57, city: "Gonda" }
{ roll: 10, name: "Nikita", age: 19, marks: 57, city: "Bareilly" }
{ roll: 18, name: "Tanya", age: 20, marks: 59, city: "Gonda" }
{ roll: 28, name: "Nandini", age: 19, marks: 58, city: "Gonda" }
{ roll: 41, name: "Dev", age: 18, marks: 55, city: "Gonda" }
```

Figure: MongoDB execution screenshot — 7. LESS THAN \$lt

8. GREATER THAN OR EQUAL TO — \$gte

The \$gte comparison operator finds students whose marks are 90 or above.

MongoDB Shell

```
db.students.find({ marks: { $gte: 90 } })
{ roll: 3, name: "Ishita", age: 20, marks: 91, city: "Varanasi" }
{ roll: 7, name: "Ritesh", age: 19, marks: 93, city: "Prayagraj" }
{ roll: 12, name: "Muskan", age: 20, marks: 95, city: "Kanpur" }
{ roll: 16, name: "Aarohi", age: 19, marks: 92, city: "Prayagraj" }
{ roll: 22, name: "Sneha", age: 19, marks: 96, city: "Lucknow" }
{ roll: 26, name: "Mehak", age: 18, marks: 90, city: "Lucknow" }
{ roll: 29, name: "Kunal", age: 18, marks: 94, city: "Kanpur" }
... 4 more document(s)
```

Figure: MongoDB execution screenshot — 8. GREATER THAN OR EQUAL TO \$gte

9. LESS THAN OR EQUAL TO — \$lte

The \$lte comparison operator finds students whose marks are 70 or below.

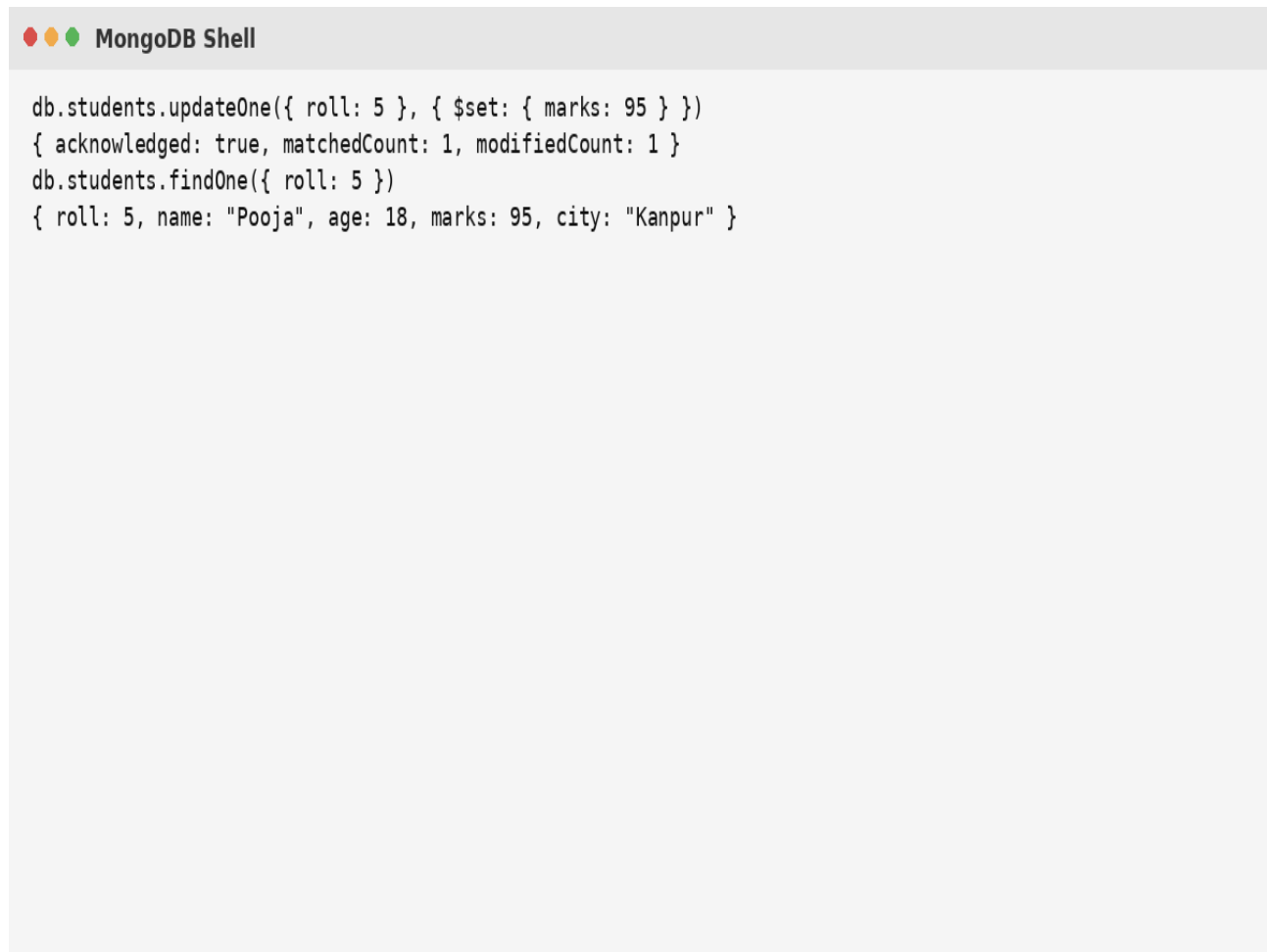
MongoDB Shell

```
db.students.find({ marks: { $lte: 70 } })
{ roll: 4, name: "Mohit", age: 19, marks: 68, city: "Jaunpur" }
{ roll: 9, name: "Saurabh", age: 20, marks: 57, city: "Gonda" }
{ roll: 10, name: "Nikita", age: 19, marks: 57, city: "Bareilly" }
{ roll: 13, name: "Rajat", age: 19, marks: 66, city: "Ayodhya" }
{ roll: 18, name: "Tanya", age: 20, marks: 59, city: "Gonda" }
{ roll: 20, name: "Komal", age: 18, marks: 70, city: "Jaunpur" }
{ roll: 21, name: "Rishabh", age: 20, marks: 64, city: "Varanasi" }
... 7 more document(s)
```

Figure: MongoDB execution screenshot — 9. LESS THAN OR EQUAL TO \$lte

10. UPDATE OPERATION

The update operation changes the marks of the student having roll number 5 to 95.

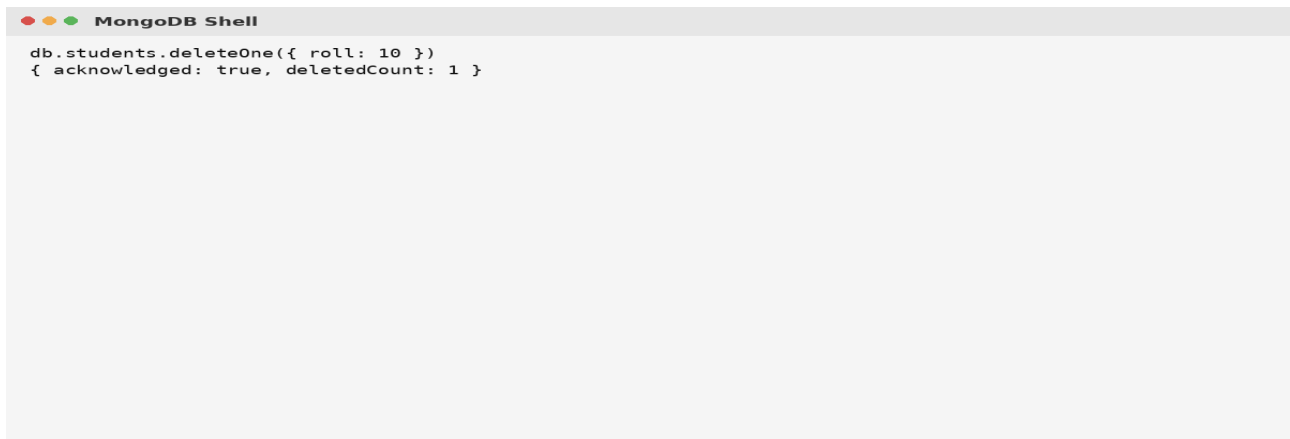
A screenshot of the MongoDB Shell interface. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "MongoDB Shell". The main area contains the following text:

```
db.students.updateOne({ roll: 5 }, { $set: { marks: 95 } })
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
db.students.findOne({ roll: 5 })
{ roll: 5, name: "Pooja", age: 18, marks: 95, city: "Kanpur" }
```

Figure: MongoDB execution screenshot — 10. UPDATE OPERATION

11. DELETE ONE OPERATION

The delete-one operation removes the document matching roll number 10.

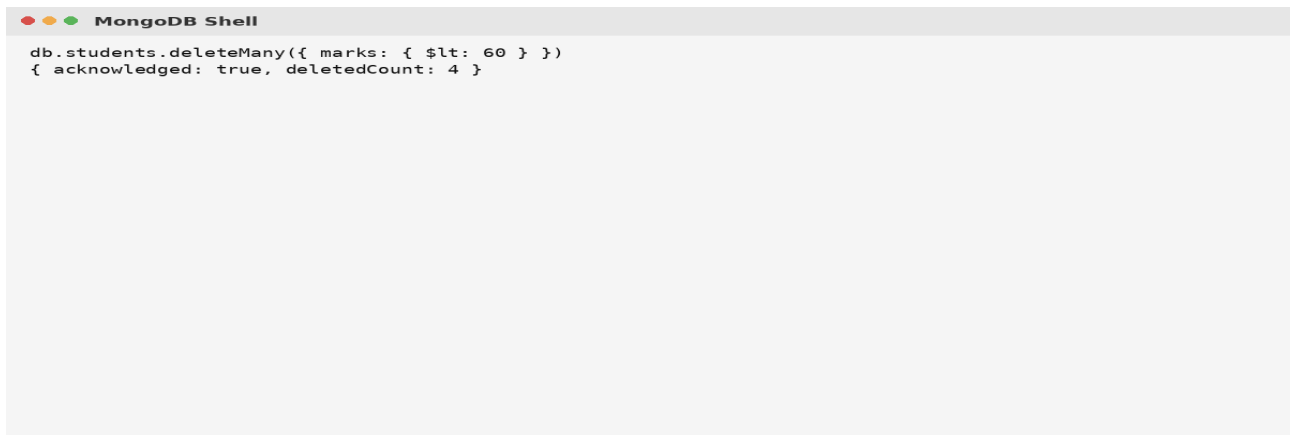
A screenshot of the MongoDB Shell interface. The title bar at the top reads "MongoDB Shell" with three colored window control buttons (red, yellow, green) on the left. The main area of the shell displays two lines of text: the first line is the command `db.students.deleteOne({ roll: 10 })` and the second line is the output `{ acknowledged: true, deletedCount: 1 }`.

```
db.students.deleteOne({ roll: 10 })
{ acknowledged: true, deletedCount: 1 }
```

Figure: MongoDB execution screenshot — 11. DELETE ONE OPERATION

12. DELETE MANY OPERATION

The delete-many operation removes all students having marks below 60.

A screenshot of the MongoDB Shell interface. The title bar at the top reads "MongoDB Shell" with three colored window control buttons (red, yellow, green) on the left. The main area of the shell displays two lines of text: the first line is the command `db.students.deleteMany({ marks: { $lt: 60 } })` and the second line is the output `{ acknowledged: true, deletedCount: 4 }`.

```
MongoDB Shell
db.students.deleteMany({ marks: { $lt: 60 } })
{ acknowledged: true, deletedCount: 4 }
```

Figure: MongoDB execution screenshot — 12. DELETE MANY OPERATION

13. AND OPERATION — \$and

The \$and logical operator returns students satisfying both conditions: marks greater than 80 and age equal to 19.

●●● MongoDB Shell

```
db.students.find({ $and: [{ marks: { $gt: 80 } }, { age: 19 }] })
{ roll: 1, name: "Riya", age: 19, marks: 82, city: "Kanpur" }
{ roll: 7, name: "Ritesh", age: 19, marks: 93, city: "Prayagraj" }
{ roll: 16, name: "Aarohi", age: 19, marks: 92, city: "Prayagraj" }
{ roll: 19, name: "Vikas", age: 19, marks: 87, city: "Lucknow" }
{ roll: 22, name: "Sneha", age: 19, marks: 96, city: "Lucknow" }
{ roll: 31, name: "Varun", age: 19, marks: 83, city: "Jaunpur" }
{ roll: 40, name: "Payal", age: 19, marks: 91, city: "Lucknow" }
... 2 more document(s)
```

Figure: MongoDB execution screenshot — 13. AND OPERATION \$and

14. OR OPERATION — \$or

The \$or logical operator returns students whose city is either Lucknow or Kanpur.

MongoDB Shell

```
db.students.find({ $or: [{ city: "Lucknow" }, { city: "Kanpur" }] })
{ roll: 1, name: "Riya", age: 19, marks: 82, city: "Kanpur" }
{ roll: 2, name: "Arnav", age: 18, marks: 74, city: "Lucknow" }
{ roll: 5, name: "Pooja", age: 18, marks: 86, city: "Kanpur" }
{ roll: 8, name: "Kavya", age: 18, marks: 88, city: "Lucknow" }
{ roll: 11, name: "Abhishek", age: 18, marks: 84, city: "Lucknow" }
{ roll: 12, name: "Muskan", age: 20, marks: 95, city: "Kanpur" }
{ roll: 15, name: "Deepak", age: 20, marks: 76, city: "Lucknow" }
... 15 more document(s)
```

Figure: MongoDB execution screenshot — 14. OR OPERATION \$or

15A. SORT OPERATION — ASCENDING

The sort operation arranges students by marks from lowest to highest. Ascending order uses 1.

MongoDB Shell

```
db.students.find().sort({ marks: 1 })
{ roll: 41, name: "Dev", age: 18, marks: 55, city: "Gonda" }
{ roll: 9, name: "Saurabh", age: 20, marks: 57, city: "Gonda" }
{ roll: 10, name: "Nikita", age: 19, marks: 57, city: "Bareilly" }
{ roll: 28, name: "Nandini", age: 19, marks: 58, city: "Gonda" }
{ roll: 18, name: "Tanya", age: 20, marks: 59, city: "Gonda" }
{ roll: 46, name: "Harshit", age: 19, marks: 60, city: "Ayodhya" }
{ roll: 37, name: "Ankit", age: 19, marks: 62, city: "Lucknow" }
... 43 more document(s)
```

Figure: MongoDB execution screenshot — 15A. SORT OPERATION ASCENDING

15B. SORT OPERATION — DESCENDING

The sort operation arranges students by marks from highest to lowest. Descending order uses -1.

MongoDB Shell

```
db.students.find().sort({ marks: -1 })
{ roll: 43, name: "Arjun", age: 19, marks: 98, city: "Lucknow" }
{ roll: 36, name: "Shivani", age: 20, marks: 97, city: "Prayagraj" }
{ roll: 22, name: "Sneha", age: 19, marks: 96, city: "Lucknow" }
{ roll: 12, name: "Muskan", age: 20, marks: 95, city: "Kanpur" }
{ roll: 29, name: "Kunal", age: 18, marks: 94, city: "Kanpur" }
{ roll: 7, name: "Ritesh", age: 19, marks: 93, city: "Prayagraj" }
{ roll: 49, name: "Sonam", age: 19, marks: 93, city: "Gorakhpur" }
... 43 more document(s)
```

Figure: MongoDB execution screenshot — 15B. SORT OPERATION DESCENDING

16. LIMIT OPERATION

The limit operation displays only the first five documents returned by the query.

●●● MongoDB Shell

```
db.students.find().limit(5)
{ roll: 1, name: "Riya", age: 19, marks: 82, city: "Kanpur" }
{ roll: 2, name: "Arnav", age: 18, marks: 74, city: "Lucknow" }
{ roll: 3, name: "Ishita", age: 20, marks: 91, city: "Varanasi" }
{ roll: 4, name: "Mohit", age: 19, marks: 68, city: "Jaunpur" }
{ roll: 5, name: "Pooja", age: 18, marks: 86, city: "Kanpur" }
```

Figure: MongoDB execution screenshot — 16. LIMIT OPERATION

17. SEARCH BY CITY

The city search finds all students whose city is Lucknow.

●●● MongoDB Shell

```
db.students.find({ city: "Lucknow" })
{ roll: 2, name: "Arnav", age: 18, marks: 74, city: "Lucknow" }
{ roll: 8, name: "Kavya", age: 18, marks: 88, city: "Lucknow" }
{ roll: 11, name: "Abhishek", age: 18, marks: 84, city: "Lucknow" }
{ roll: 15, name: "Deepak", age: 20, marks: 76, city: "Lucknow" }
{ roll: 19, name: "Vikas", age: 19, marks: 87, city: "Lucknow" }
{ roll: 22, name: "Sneha", age: 19, marks: 96, city: "Lucknow" }
{ roll: 26, name: "Mehak", age: 18, marks: 90, city: "Lucknow" }
... 6 more document(s)
```

Figure: MongoDB execution screenshot — 17. SEARCH BY CITY

18. COUNT DOCUMENTS

The count operation counts the documents currently present in the collection after the delete operations.

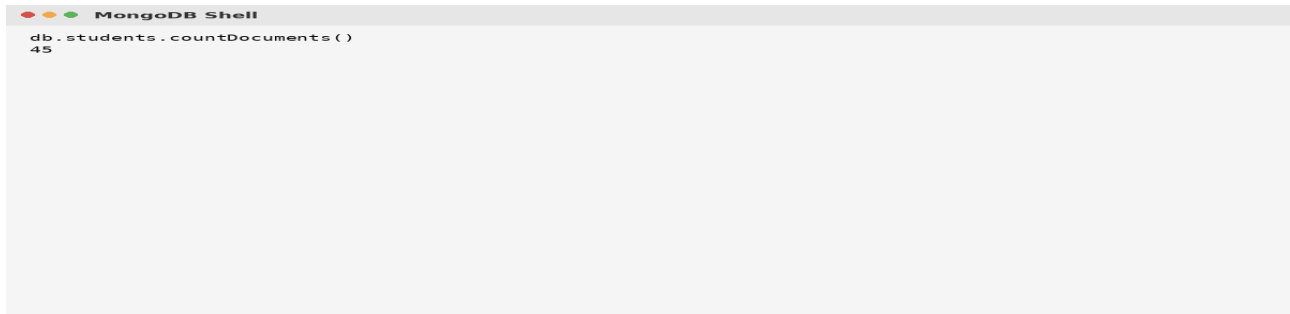


Figure: MongoDB execution screenshot — 18. COUNT DOCUMENTS

MONGODB OPERATORS USED

The project demonstrates comparison and logical operators along with common result-processing methods used for MongoDB queries.

Operator / Method	Meaning	Example / Use
\$eq	Equal to	marks: {\$eq: 88}
\$gt	Greater than	marks: {\$gt: 85}
\$lt	Less than	marks: {\$lt: 60}
\$gte	Greater than or equal to	marks: {\$gte: 90}
\$lte	Less than or equal to	marks: {\$lte: 70}
\$and	All specified conditions must be true	\$and: [...]
\$or	At least one specified condition can be true	\$or: [...]
sort()	Sorts query results	Ascending / descending order
limit()	Limits the number of returned documents	First 5 documents
countDocuments()	Counts documents matching the collection query	Current document count

PROJECT WORKFLOW

The workflow followed in the project was:

1. Create/select the Students database.
2. Create the students collection.
3. Insert the 50 student documents.
4. Retrieve and display student records.
5. Apply comparison operators for filtering.
6. Update and delete selected records.
7. Apply AND and OR logical operators.
8. Sort and limit the returned results.
9. Search records by city.
10. Count the remaining documents after deletion.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how ascending and descending order work, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte are used, and how logical operators such as \$and and \$or help in filtering data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.

PROJECT DETAILS

Student Name	Diwakar Yadav
Roll No.	34
Class	BCA DS23
Department	BCA (DS & AI)
Teacher	Mr. Harendra Singh Rajpoot
Project	Student Management System Using MongoDB
Database	Students
Collection	students
Initial Records	50
Operations Demonstrated	Create, insert, display, filter, update, delete, sort, limit, search, count
Academic Session	2026/2027

End of Project Report